

Lightweight Network Anomaly Detection — Lab Testing Manual

Detector (victim): Windows 10 VM · Attacker: Kali Linux VM · Generated from the project repo

1. Lab topology

A two-VM lab on one host (VirtualBox or VMware). Both VMs must be on the **same virtual network** so attack traffic actually crosses the victim NIC the detector is sniffing. Use **Bridged** networking (recommended) or a shared Host-Only/NAT network with working ping between the VMs.

Role	System	Purpose
Victim / Detector	Windows 10 VM	Runs the FastAPI backend that captures its own NIC, stores alerts in SQLite, and serves the React dashboard
Attacker	Kali Linux VM	Generates the attacks (nmap, hping3, Scapy) toward the Windows VM IP

- **Why VMs make this work:** VM NICs are virtual *Ethernet*. The Scapy capture bug seen on a real managed-mode Wi-Fi adapter (RadioTap decode failure) does not occur here, so live capture is reliable without monitor mode.
- Example addressing: Windows VM **192.168.1.50/24**, Kali VM **192.168.1.60/24**. Replace every IP below with your actual lab addresses.
- The free alive-test smoke script (*scripts/alive_test.py*) can also be run on the Windows VM to sanity-check the pipeline before using real attacks.

2. Recommended (best) testing settings

Create the project root `.env` file (copy `.env.example`) and set the values below. The **lab values** lower thresholds so attacks are detected within seconds; the defaults are more realistic for production.

```
# C:\nad\.env -- recommended lab settings
NETWORK_INTERFACE=Ethernet           # exact NIC name from ipconfig (Scapy-friendly)
PORT_SCAN_THRESHOLD=10                # distinct ports from one source in 5s (default 20)
PORT_SCAN_WINDOW=5
PORT_SCAN_COOLDOWN=30
CONNECTION_THRESHOLD=50               # new connections in 10s (default 100)
CONNECTION_WINDOW=10
CONNECTION_COOLDOWN=30
TRAFFIC_BASELINE_WINDOW=60
TRAFFIC_SPIKE_MULTIPLIER=3.0          # burst must exceed 3x baseline
TRAFFIC_MIN_SAMPLES=4                # baseline seconds to learn (default 5)
TRAFFIC_COOLDOWN=30
DNS_THRESHOLD=30                     # DNS queries in 10s (default 50)
DNS_WINDOW=10
DNS_LONG_DOMAIN_LENGTH=40            # characters (default 50)
DNS_COOLDOWN=30
SUSPICIOUS_DESTINATIONS=203.0.113.5 # test IP for the outbound scenario (TEST-NET-3)
MAX_TRACKING_AGE=120
API_HOST=0.0.0.0
API_PORT=8000
DATABASE_PATH=./data/alerts.db
```

Setting	Purpose	Default	Lab	Notes
PORT_SCAN_THRESHOLD	Port-scan alert	20	10	distinct TCP/UDP ports from one source within PORT_SCAN_WINDOW
CONNECTION_THRESHOLD	Connection burst	100	50	new connections from one source within CONNECTION_WINDOW
TRAFFIC_MIN_SAMPLES	Traffic spike	5	4	baseline seconds; spike = current pps > 3x baseline average
DNS_THRESHOLD / DNS_LONG_DOMAIN_LENGTH	DNS anomaly	50 / 50	30 / 40	query rate within window, or a single over-long query name
SUSPICIOUS_DESTINATIONS	Outbound alert	(empty)	one IP	any request to these IPs raises SUSPICIOUS_OUTBOUND (LOW)
TRUSTED_NETWORKS	normal-traffic baseline	10/8,172.16/12,192.168/16	same	keep the private ranges; alerts focus on the unusual

Cooldowns (30–60 s) suppress repeat alerts for the same source. Run different scenarios at least a minute apart, or stop the backend, delete data/alerts.db, and restart for a clean slate.

3. Windows 10 VM — detector setup

3.1 Install prerequisites

- Install **Python 3.12+ (x64)** from python.org; tick “Add Python to PATH”.
- Install **Npcap** from npcap.com (default options). **Required** — Scapy cannot capture on Windows without it.
- Install **Node.js 18+ (LTS)** from nodejs.org for the dashboard.

3.2 Put the project in the VM and install dependencies

```
cd C:\nad
copy .env.example .env
python -m venv backend\venv
backend\venv\Scripts\activate
pip install -r backend\requirements.txt
```

3.3 Find the exact NIC name and put it in .env

With the venv activated and **Npcap installed** (admin prompt is safest):

```
python -m app.capture.sniffer --list-interfaces
ipconfig
```

Put the exact adapter name (e.g. Ethernet, Wi-Fi) into .env as NETWORK_INTERFACE.

Easier alternative: the backend auto-selects a usable NIC when NETWORK_INTERFACE is left unset or points to an adapter that does not exist. You may then simply omit the variable in a lab.

3.4 Windows Firewall (lab only)

To reach the API and dashboard from the Kali VM or host, open TCP 8000 and 5173 — or for the duration of the lab simply **disable** the three firewall profiles in the VM and re-enable them after testing.

4. Start the stack in the VM

4.1 Backend (run as Administrator)

```
cd C:\nad
backend\venv\Scripts\activate
set NETWORK_INTERFACE=Ethernet
python -m uvicorn app.main:app --app-dir backend --host 0.0.0.0 --port 8000
```

Confirm capture is live (expect “running”: true):

```
curl.exe http://127.0.0.1:8000/api/capture/status
```

4.2 Dashboard (second terminal)

```
cd C:\nad\frontend
npm install
npm run dev
```

Open **http://localhost:5173** in the VM (or **http://<VM-IP>:5173** from the host/Kali). It polls the API every 3 s.

5. Kali VM — attacker setup

```
ping 192.168.1.50 # must succeed
# tools already present: nmap, hping3, dnsutils(dig), python3
sudo apt install -y python3-scapy # only if missing
```

6. Attack and verify scenarios

Run each scenario from Kali toward the Windows IP, wait ~2 s, then check the alert in the dashboard or via the API (section 7).

6.1 Port scan → PORT_SCAN (HIGH)

```
nmap -sS -Pn -n -p1-30 --min-rate 500 192.168.1.50
```

More than 10 distinct TCP ports from the Kali IP within 5 s triggers the alert. Add -sU with separate runs for UDP ports if you want a UDP scan.

6.2 Connection burst → CONNECTION_ANOMALY (MEDIUM)

```
hping3 -S -p 8080 -c 2000 --fast 192.168.1.50
```

Over 50 new connections from the source within a 10 s window triggers the alert.

6.3 TCP flood / traffic spike → TRAFFIC_ANOMALY (MEDIUM)

```
hping3 -p 8000 --flood --rand-source 192.168.1.50 # let it run 8-10 s
```

Once the packet rate exceeds 3x the learned baseline in a one-second bucket, TRAFFIC_ANOMALY fires.

6.4 DNS query storm → DNS_ANOMALY (MEDIUM)

The detector parses DNS *request* packets on port 53 even if no DNS server answers, so crafted queries from Kali are enough:

```
sudo python3 - <<'EOF'
from scapy.all import IP, UDP, DNS, DNSQR, send
for i in range(35):
    send(IP(src="192.168.1.60", dst="192.168.1.50")
        / UDP(sport=2000+i, dport=53)
        / DNS(rd=1, qd=DNSQR(qname="%d.attack-lab.example" % i)), verbose=0)
```

EOF

35 queries in 10 s > threshold 30. To also trigger the **long-domain** rule, send one query with a 45+ character name: `qname="a"*45+".example.com"`.

6.5 Suspicious outbound contact → SUSPICIOUS_OUTBOUND (LOW)

From the **Windows VM** (simulating a compromised host phoning home):

```
curl.exe --max-time 5 http://203.0.113.5
```

Any request from the victim to an IP listed in `SUSPICIOUS_DESTINATIONS` raises `SUSPICIOUS_OUTBOUND` (LOW).

Scenario	Expected alert type	Severity	Quick check
6.1 Port scan	PORT_SCAN	HIGH	source = Kali IP, destination = Windows IP
6.2 Connection burst	CONNECTION_ANOMALY	MEDIUM	many connections from Kali IP in 10 s
6.3 Flood	TRAFFIC_ANOMALY	MEDIUM	pps spikes above 3x baseline
6.4 DNS storm	DNS_ANOMALY	MEDIUM	35+ queries in 10 s (or a 45+ char name)
6.5 Outbound	SUSPICIOUS_OUTBOUND	LOW	destination = the suspicious IP

7. Reading the results

- Dashboard `http://localhost:5173` — StatsOverview counters, AlertTable (newest first) with status buttons (NEW / ACKNOWLEDGED / RESOLVED), TrafficChart, DevicesTable of source IPs.

- API — stats, alert list, capture health:

```
curl.exe http://127.0.0.1:8000/api/stats
curl.exe "http://127.0.0.1:8000/api/alerts?limit=50"
curl.exe http://127.0.0.1:8000/api/capture/status
```

- Every alert row carries `alert_type`, `severity`, `source/destination IP` and ports, protocol, description, and a JSON metadata field with the triggering evidence (e.g. unique ports counted).

8. Troubleshooting

Symptom	Cause and fix
capture/status shows <code>running=false</code>	Wrong <code>NETWORK_INTERFACE</code> name. Re-run the <code>get_windows_if_list</code> command, fix <code>.env</code> , restart the backend.
sniffer <code>PermissionError</code> in backend log	Backend not run as Administrator, or Npcap not installed. Run the cmd as admin and install Npcap.
API unreachable from Kali/host	Windows Firewall blocks 8000/5173. "Allow an app through the firewall" or disable the lab profiles.
0 alerts even though attack ran	Check capture/status <code>packet_count</code> is increasing. If yes, the attack was below threshold (use <code>--min-rate</code> / more packets, or lower the lab thresholds). If 0, capture is not seeing the NIC — fix the interface first.
Dashboard blank / no data	Backend not running on 8000, or Node dev server not started. Check both terminals.

Symptom	Cause and fix
Alerts repeat for the same event	Cooldowns are per source. Wait 30–60 s or reset by deleting data\alerts.db (backend stopped) then restarting.
SUSPICIOUS_OUTBOUND never fires	Outbound rule is about destinations. Add the target IP to SUSPICIOUS_DESTINATIONS in .env and restart.

9. Honest limits of this lab

- Detection is **threshold-based**, not semantic. A slow 2-port scan or a single quiet exfiltration packet will not alert — that is by design in v1.
- Scapy visibility is **local to the Windows VM**. It only analyzes traffic on that VM's NIC, so the Kali attacks must actually reach that NIC (working ping = good sign).
- These lab values are intentionally aggressive and will produce false positives on a busy real network; restore the defaults for production.